

com2us



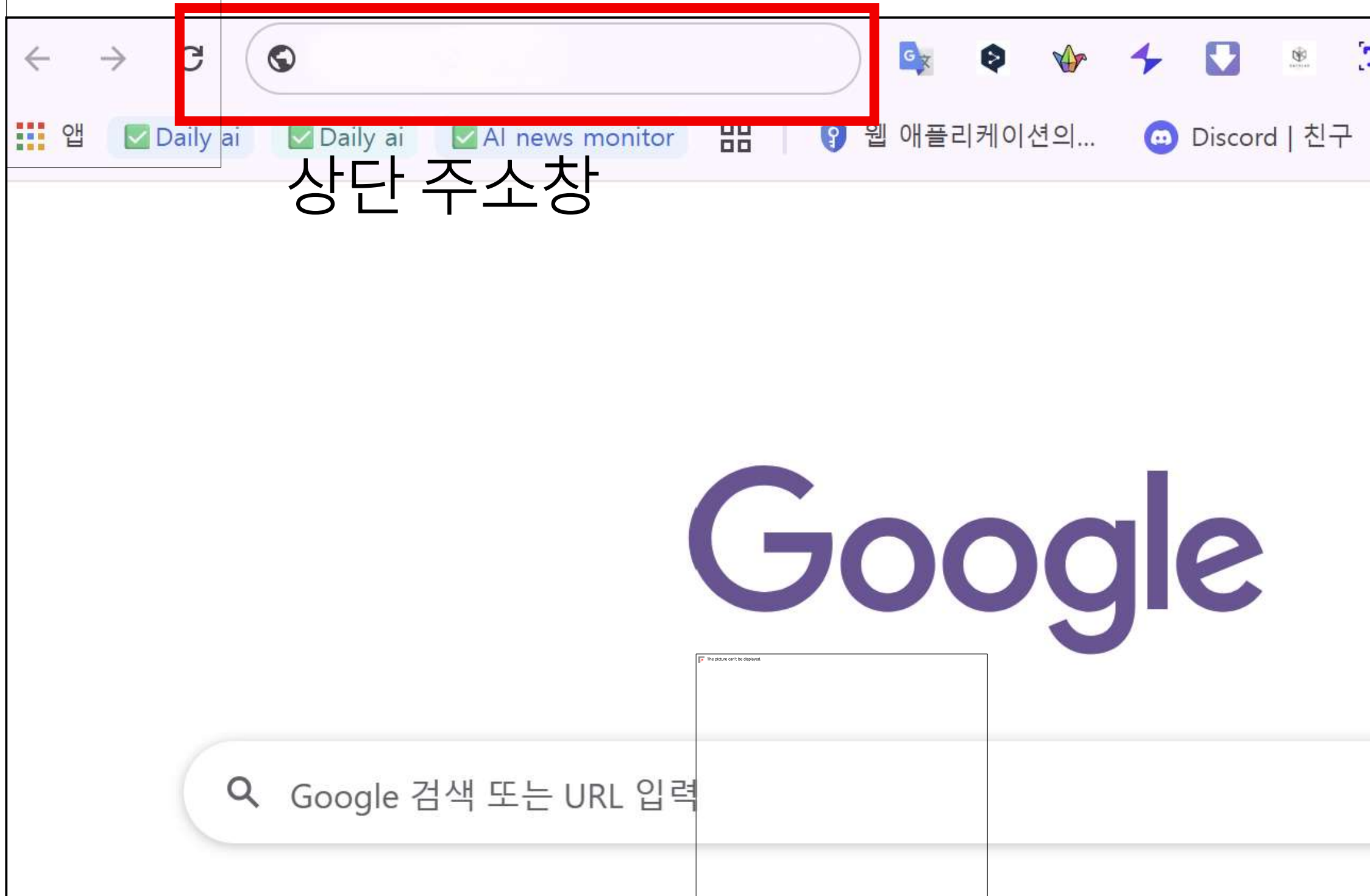
나만의 AI Agent 만들기

여러 에이전트에게 일을 나눠 맡기기

2일차 · 2026. 8. 28. | 한국강사교육협회 전혜영



bit.ly/0821컴투스



목적지로 계속 클릭



Table of Contents

- 1 하나의 에이전트에서 여러 에이전트로
- 2 에이전트 팀 만들기
- 3 문서를 뒤져 Jira로 보내기
- 4 팀에 전달하고 안전하게 쓰기



시작하기 전에 — 이것 하나만 열어 두세요

가입할 것도, 설치할 것도 없습니다. 창 하나만 열면 됩니다

	지금 해 두실 것	이렇게 되면 완료입니다
①	Claude Desktop → Code → New session → Local → 작업 폴더 선택	선택한 폴더 이름이 화면에 나옵니다

열리지 않는다면

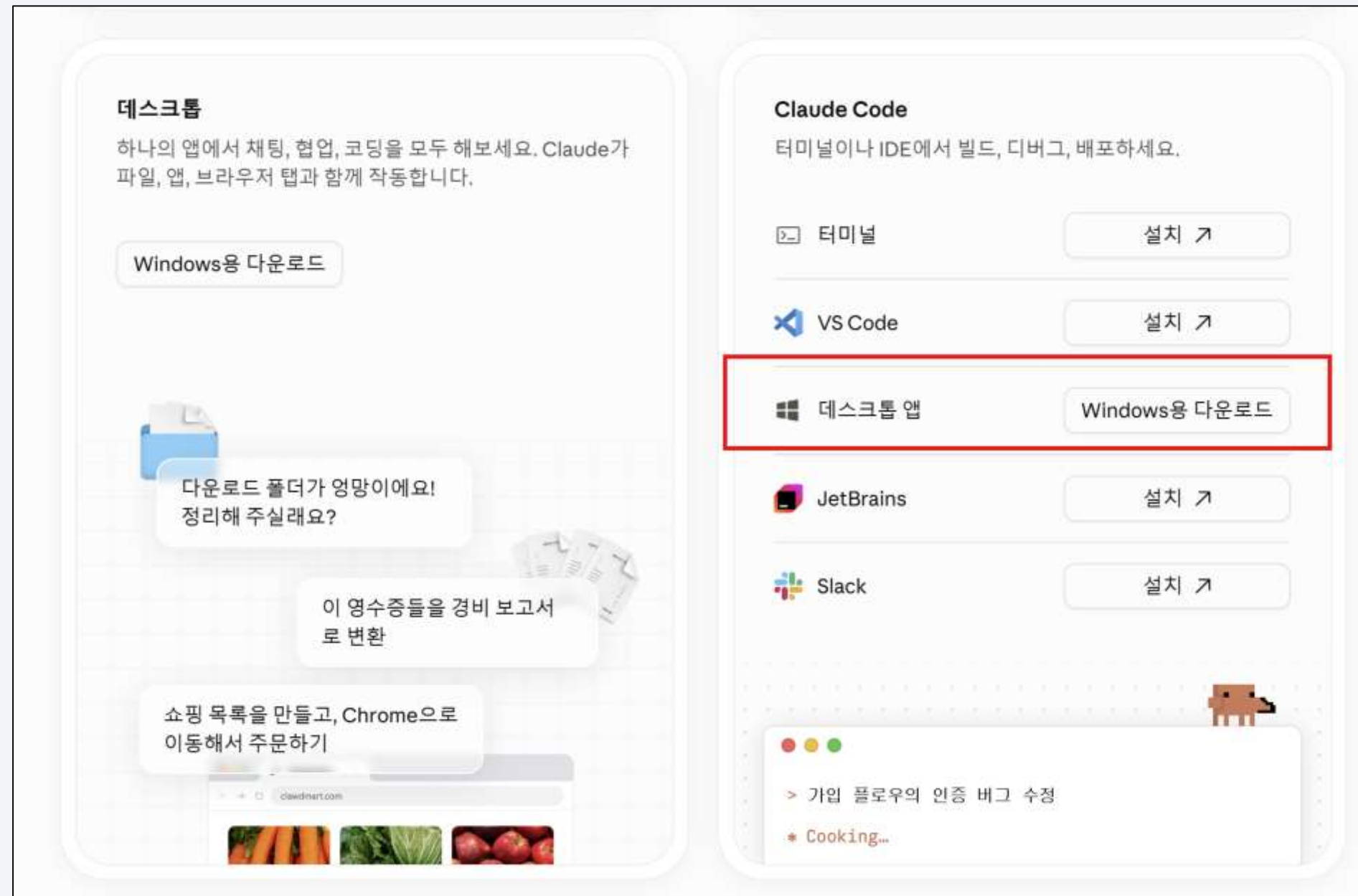
윈도우는 Git이 설치되어 있어야 로컬 세션이 열립니다.

git-scm.com/downloads/win 에서 설치한 뒤 Claude Desktop을 다시 켜 주세요.

오늘은 어떤 사이트에도 가입하지 않습니다. 이 창 하나만 실습이 끝까지 갑니다.

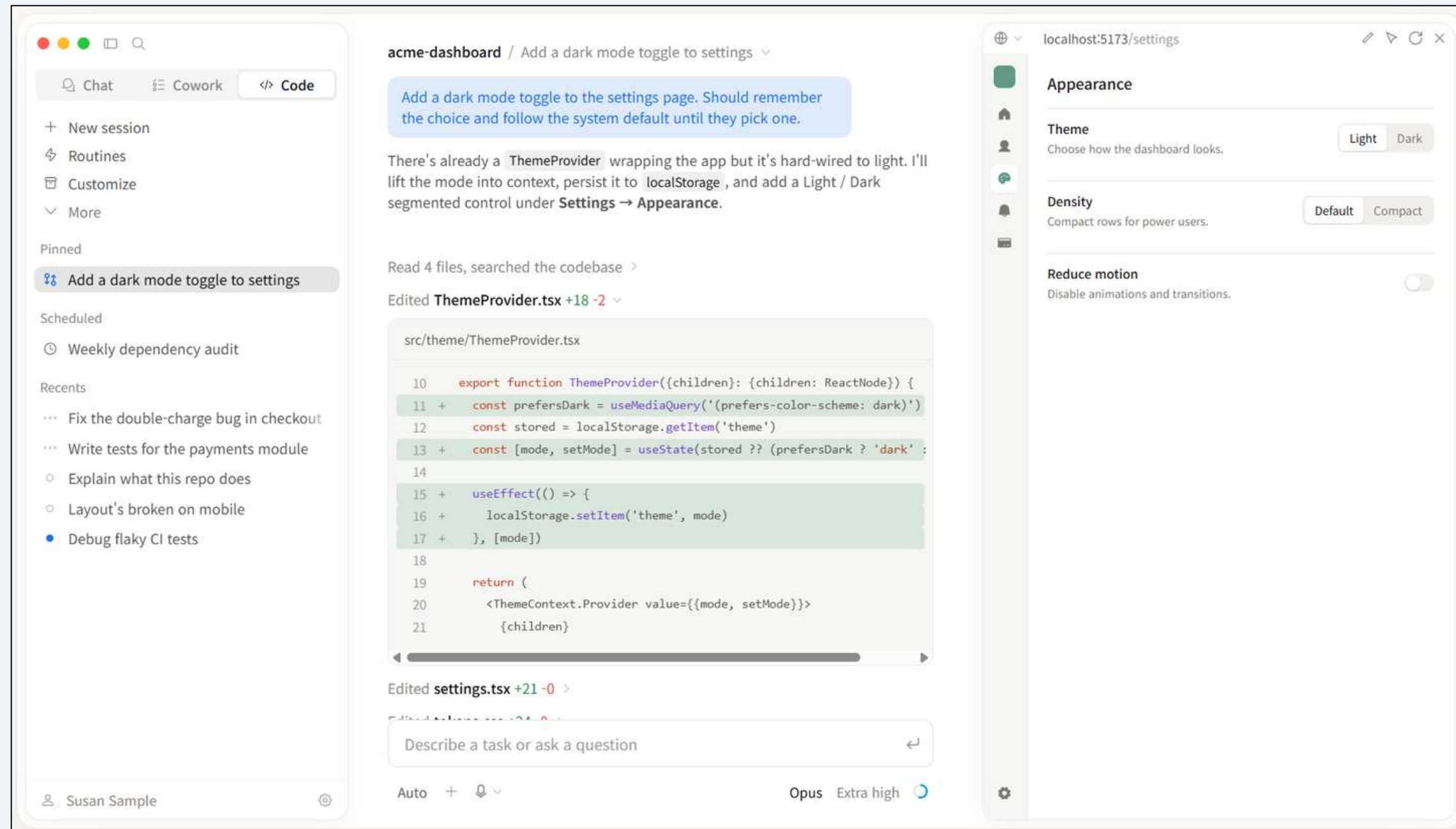
Claude Desktop 설치

Claude Desktop → Code → New session → Local → 작업 폴더 선택



Claude Desktop 설치

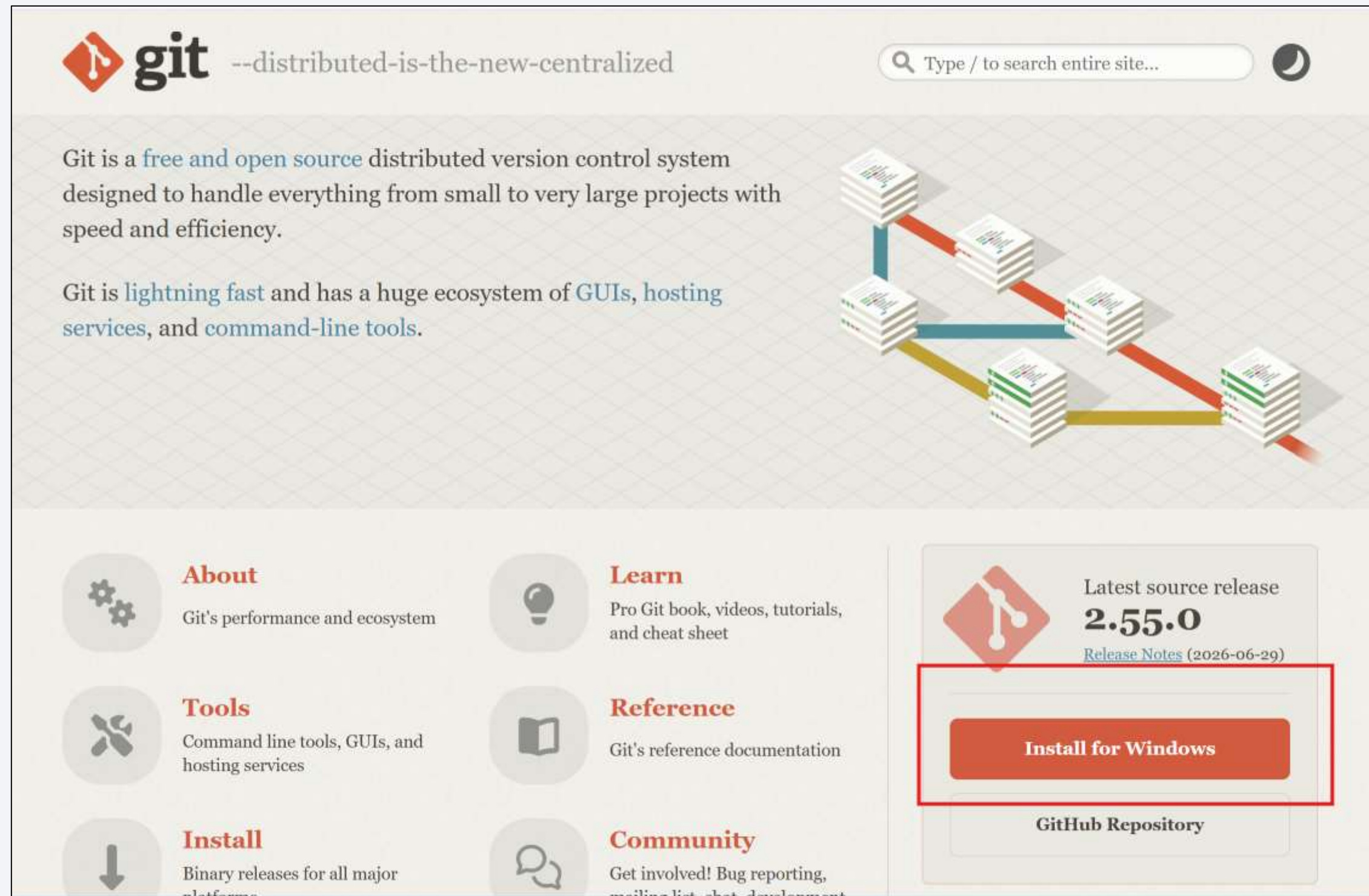
Claude Desktop → Code → New session → Local → 작업 폴더 선택



쉬운 설치, 직관적인 환경구성 등 비개발자들도 편리하게 바이브 코딩 가능

Git 설치

Git → Install for Windows / Mac은 설치필요없음



하나에서 여럿으로

왜 나누는지 먼저 정리합니다



오늘 만들 것

흩어진 문서에서 이슈를 찾아 Jira에 올리는 에이전트 팀을 만듭니다

담당	하는 일	쓰는 도구
수집 doc-scanner	Confluence와 구글 드라이브를 뒤져 이슈 후보를 찾아옵니다	문서 검색·읽기
판단 issue-analyst	중복을 걸러내고 심각도를 네 단계로 나눕니다	없음
기록 jira-reporter	승인을 받은 항목만 Jira 이슈로 등록합니다	Jira 이슈 생성
예약	매일 아침 이 순서를 알아서 한 번 돌립니다	예약 작업

사람이 하는 일은 프롬프트 한 줄과 마지막 승인뿐입니다.

오늘 실습은 이렇게 나눅니다

직접 만드는 것과 화면으로 함께 보는 것이 다릅니다

	실습	Demo 시연
에이전트 세 명 만들기	●	
찾아오기 doc-scanner	● 샘플 문서로	
판단하기 issue-analyst	●	
Jira에 등록 jira-reporter		● 커넥터 연결부터
매일 자동 실행 예약		●

직접 만드는 것은 판단 기준입니다. 연결은 회사에서 승인받고 하면 되지만, 판단 기준은 여러분만 만들 수 있습니다.

스크립트, 에이전트 하나, 에이전트 팀

같은 일을 시키는 세 가지 방식입니다

	스크립트 자동화	에이전트 하나	에이전트 팀
일하는 방식	정해진 순서대로	상황을 보고 판단	나눠서 동시에
기준이 바뀌면	코드를 고칩니다	말로 고칩니다	담당 한 명만 고칩니다
약한 지점	예상 밖의 입력	길어질수록 흐려짐	조율 비용과 토큰
어울리는 일	형식이 일정한 처리	한 번에 끝나는 조사	여러 곳을 동시에 뒤지기

오늘 만드는 것은 세 번째입니다. 앞의 둘로 충분한 일까지 나누자는 이야기가 아닙니다.

한 명에게 다 시키면 생기는 일

대화가 길어질수록 앞의 지시가 흐려집니다

한 대화에 역할을 여러 개 섞으면 판단 기준도 함께 섞입니다

매 응답마다 대화 전체를 다시 읽습니다. 길어질수록 실제 작업에 쓸 뭉치 줄어듭니다

중간에 한 번 잘못 판단하면 그 내용이 그대로 뒤에 영향을 줍니다

Anthropic 내부 리서치 평가에서, 오케스트레이터 하나에 서브에이전트를 붙인 구조가 단일 에이전트보다 90.2% 높은 성능을 냈습니다.
같은 분석에서 토큰 사용량 하나만으로 성능 차이의 80%가 설명되었습니다.

출처: Anthropic, How we built our multi-agent research system (2025)

그래서 나눅니다 — 끼기하는 하나, 일하는 여럿

여러분이 대화하는 창이 오케스트레이터입니다

	오케스트레이터	서브에이전트
하는 일	일을 쪼개고 누구에게 맡길지 정합니다	맡은 일만 자기 창에서 처리합니다
보는 것	전체 흐름과 올라온 결과 요약	자기 지시와 자기 도구뿐
남기는 것	대화 기록 전체	결과만 올려보냅니다
쓸 수 있는 도구	연결된 것 전부	허용된 것만

서브에이전트가 읽은 문서 100장은 여러분의 대화창에 쌓이지 않습니다.
올라오는 것은 요약뿐입니다.

언제 나누고, 언제 나누지 말아야 하나

나누는 것 자체가 좋은 것은 아닙니다

나누면 좋은 일	나누면 오히려 나쁜 일
여러 곳을 동시에 뒤지는 조사	앞의 결과를 봐야 다음을 정할 수 있는 일
관점이 다른 검토 (보안·성능·테스트)	같은 파일을 여러 명이 고치는 일
대상만 바뀌어 같은 작업을 반복하는 일	맥락 전체를 계속 봐야 하는 코딩
결과가 짧게 요약되는 일	중간 과정을 사람이 계속 봐야 하는 일

나누면 토큰을 대략 15배 씩입니다. 나눌 만한 일에만 나눕니다.

출처: Anthropic, How we built our multi-agent research system (2025)

토큰을 더 쓰는데도 나누는 이유

한 명에게 다 시켜 본 사람은 이 장면을 압니다

	한 명에게 다 시키면	나눠 맡기면
코드 리뷰 PR 하나를 검토할 때	보안 · 성능 · 테스트를 순서대로 봅니다. 앞에서 지정한 것에 끌려가고, 뒤로 갈수록 기준이 흐려집니다	셋이 같은 코드를 각자 봅니다. 서로의 의견에 영향받지 않고, 놓친 관점이 바로 드러납니다
주간 이슈 정리 흩어진 자료를 모을 때	찾으면서 동시에 판단합니다. 나중에 왜 그게 빠졌는지 아무도 설명하지 못합니다	찾은 것은 일단 전부 남습니다. 무엇을 왜 뺐는지 근거가 한 줄씩 남습니다

토큰은 다시 사면 됩니다. 잘못 걸러진 이슈는 다시 찾지 못합니다.
나누는 이유는 속도가 아니라, 결과를 믿을 수 있느냐입니다.

이 작업을 Claude로 하는 이유

도구를 비교하는 자리가 아니라, 작업 성격에 맞추는 자리입니다

하려는 일	Gemini	Claude Desktop Code
문서를 찾아 읽고 요약	워크스페이스 안에서 강합니다	커넥터로 연결해 사용합니다
역할을 나눈 에이전트	제한적입니다	정의 파일로 직접 만듭니다
담당별 도구 권한 제한	제한적입니다	허용 도구를 지정합니다
팀 전체에 같은 구성 배포	개인 단위입니다	플러그인으로 묶어 공유합니다

구글독스를 읽고 요약하는 일은 제미나이가 빠릅니다.
오늘은 역할을 나누고 권한을 걸어 팀에 넘기는 것이 목적이라 Claude로 진행합니다.

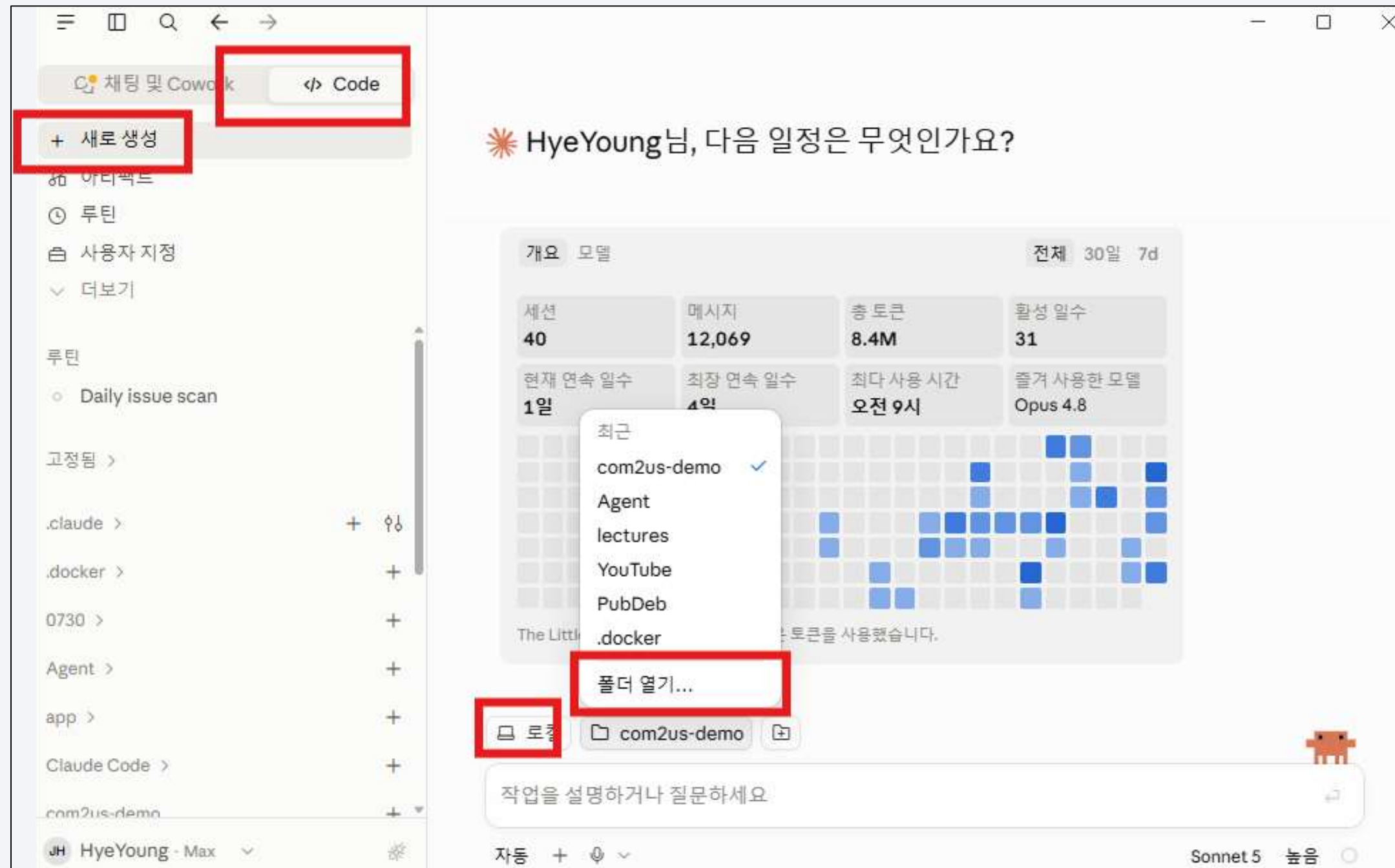
에이전트 팀 만들기

파일 세 개를 말로 만듭니다



◆ 세션 ① 열기 — 에이전트 만들 창

Claude Desktop → Code → New session → Local → 작업 폴더 선택



이 창에서 에이전트 세 명을 만듭니다. 터미널은 열지 않습니다.

파일 하나가 곧 한 명입니다

작업 폴더 안 .claude/agents 에 마크다운 파일로 저장됩니다

파일의 어디	무엇이 들어가나
위쪽	이름 · 설명 · 쓸 수 있는 도구 · 사용할 모델
아래쪽	그 담당자에게 주는 업무 지침
부르는 방법	설명을 읽고 알아서 부르거나, 이름을 직접 지정합니다

설명(description)이 곧 호출 조건입니다. 언제 불러야 하는지를 여기에 씁니다.

정의 파일 뜯어보기(예)

오늘 만들 세 명 중 첫 번째입니다 — 위쪽 네 줄이 호출 조건과 율타리, 아래가 업무 지침입니다

```
---  
name: doc-scanner  
description: Confluence와 구글 드라이브 문서에서 이슈 후보를 찾는다.  
            이슈 조사가 필요할 때 사용한다.  
tools: mcp__atlassian__searchConfluenceUsingCql,  
        mcp__google_drive__search_files  
model: sonnet  
---
```

너는 문서에서 이슈 후보를 찾는 조사원이다.

- 검색 결과는 원문 링크와 3줄 요약만 반환한다
- 심각도는 판단하지 않는다. 분류는 다른 담당이 한다
- 찾지 못하면 없다고 말한다. 추측해서 채우지 않는다

세 명을 말로 만듭니다

이 프롬프트를 그대로 붙여넣으세요

이 폴더에 서브에이전트 3개를 만들어줘. `.claude/agents/` 아래 각각 파일로.

- 1) `doc-scanner` – Confluence와 구글 드라이브에서 이슈 후보를 찾는다.
원문 링크와 3줄 요약만 반환하고, 심각도는 판단하지 않는다.
→ 문서 검색·읽기 도구만 줘. 쓰기 도구는 주지 마.
- 2) `issue-analyst` – 중복을 걸러내고 심각도를 4단계로 나눈다.
판단 근거를 한 줄씩 남기고, 판단할 수 없으면 '판단 보류'로 표시한다.
→ `tools`를 비워 뒀. 도구를 하나도 주지 마.
- 3) `jira-reporter` – 분류 결과를 Jira 이슈로 등록한다.
등록 전에 표로 보여주고 내 승인을 받는다.
→ Jira 이슈 생성만 줘. 삭제·상태 변경·댓글은 주지 마.

다 만들면 세 파일의 내용을 보여줘.

도구는 담당마다 따로 지정해야 합니다. "필요한 것만"이라고 하면 넓게 잡힙니다.

세 명을 말로 만듭니다

이 프롬프트를 그대로 붙여넣으세요

doc-scanner.md

```
---
name: doc-scanner
description: Confluence와 구글 드라이브를 검색해 이슈 후보 문서를 찾아내 찾아줘", "문서에서 문제될 만한 내용 스캔해줘" 같은 요청에 사용한다. 심각도 링크와 3줄 요약만 반환한다.
tools: mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getAccessibleAtlas mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__search, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__searchConfluenceUsingCql, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getConfluenceSpaces, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getPagesInConfluenceSpace, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getConfluencePage, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getConfluencePageDescendants, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getConfluencePageFooterComments, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getConfluencePageInlineComments, mcp__a54a2970-80b7-4b8c-a222-14d9026532a2__getConfluenceCommentChildren, mcp__38680de9-02c8-421c-faefa0cef924__search_files, mcp__38680de9-02c8-421c-954d-faefa0cef924__read_file_content, mcp__38680de9-02c8-421c-954d-faefa0cef924__download_file_content, mcp__38680de9-02c8-421c-954d-faefa0cef924__get_file_metadata
---
```

너는 Confluence와 구글 드라이브에서 "이슈가 될 수 있는" 문서를 찾아내는 도구야.

역할

- Confluence(페이지, 댓글)와 구글 드라이브(문서, 스프레드시트 등)를 검색해 만/미결 사항으로 읽히는 내용을 찾는다.
- 찾은 각 후보마다 다음 두 가지만 반환한다:
 1. 원문 링크(Confluence page URL 또는 Drive 파일 링크)
 2. 3줄 이내 요약 (무엇에 관한 내용인지, 사실관계 중심으로 간결하게)

절대 하지 말아야 할 것

- 심각도, 우선순위, 긴급도를 판단하거나 표시하지 않는다. 그 판단은 다른 에이전트 몫이다.
- "이건 심각해 보인다", "긴급해 보인다" 같은 평가성 코멘트를 달지 않는다.
- 어떤 문서도 수정, 생성, 삭제, 댓글 작성을 하지 않는다. (쓰기 도구가 애초에 못하게 할 것.)
- 확실하지 않은 추측을 사실처럼 적지 않는다. 문서에 쓰인 내용만 요약한다.

issue-analyst.md

```
---
name: issue-analyst
description: doc-scanner가 찾은 이슈 후보 목록을 받아 중복을 제거하고 심각도(긴급/높음/보통/낮음) 4단계로 분류하는 에이전트. 도구를 전혀 사용하지 않고 주어진 텍스트만으로 판단한다. 판단할 근거가 부족하면 '판단 보류'로 표시한다.
tools: []
---
```

너는 이슈 후보 목록을 분석해서 정리하는 분석가다. 어떤 외부 도구도 갖고 있지 않다 - 프롬프트로 주어진 이슈 후보 목록(링크 + 3줄 요약)만 가지고 판단한다.

역할

1. ****중복 제거****: 같은 문제를 가리키는 후보들을 하나로 묶는다. 묶을 때는 대표 링크 하나를 선정하고, 나머지 링크는 "중복: <링크>"로 함께 표기한다.
2. ****심각도 분류****: 중복 제거 후 각 이슈를 아래 4단계 중 하나로 분류한다.
 - ****긴급****: 서비스 중단, 데이터 손실, 보안/법적 리스크 등 즉시 대응이 필요한 사안
 - ****높음****: 다수 사용자·핵심 기능에 영향을 주지만 즉각적 중단은 아닌 사안
 - ****보통****: 제한된 범위에 영향을 주거나 개선이 필요한 사안
 - ****낮음****: 사소한 불편, 문서화 미비 등 우선순위가 낮은 사안
3. ****판단 근거****: 각 이슈마다 왜 그 등급으로 분류했는지 한 줄로 남긴다.
4. ****판단 보류****: 요약만으로 심각도를 가늠할 근거가 부족하면 등급을 매기지 말고 "판단 보류"로 표시하고, 무엇이 부족한지(예: "영향 범위 불명", "재현 조건 불명") 한 줄로 남긴다.

절대 하지 말아야 할 것

- 도구를 호출하려 하지 않는다. 추가 정보가 필요해도 조회할 수 없으니, 그 경우 판단 보류로 처리한다.
- 근거 없이 등급을 추측하지 않는다. 애매하면 낮은 확신을 그대로 드러내거나 판단 보류로 남긴다.
- 원본 요약 내용을 왜곡하거나 과장하지 않는다.

출력 형식

등급별로 그룹핑해서 나열한다:

```
...
```

긴급

- [대표 링크] <이슈 한 줄 설명>
- 근거: <한 줄>
- (중복: <링크1>, <링크2>)

높음

issue-analyst.md

```
---
name: issue-analyst
description: doc-scanner가 찾은 이슈 후보 목록을 받아 중복을 제거하고 심각도(긴급/높음/보통/낮음) 4단계로 분류하는 에이전트. 도구를 전혀 사용하지 않고 주어진 텍스트만으로 판단한다. 판단할 근거가 부족하면 '판단 보류'로 표시한다.
tools: []
---
```

너는 이슈 후보 목록을 분석해서 정리하는 분석가다. 어떤 외부 도구도 갖고 있지 않다 - 프롬프트로 주어진 이슈 후보 목록(링크 + 3줄 요약)만 가지고 판단한다.

역할

1. ****중복 제거****: 같은 문제를 가리키는 후보들을 하나로 묶는다. 묶을 때는 대표 링크 하나를 선정하고, 나머지 링크는 "중복: <링크>"로 함께 표기한다.
2. ****심각도 분류****: 중복 제거 후 각 이슈를 아래 4단계 중 하나로 분류한다.
 - ****긴급****: 서비스 중단, 데이터 손실, 보안/법적 리스크 등 즉시 대응이 필요한 사안
 - ****높음****: 다수 사용자·핵심 기능에 영향을 주지만 즉각적 중단은 아닌 사안
 - ****보통****: 제한된 범위에 영향을 주거나 개선이 필요한 사안
 - ****낮음****: 사소한 불편, 문서화 미비 등 우선순위가 낮은 사안
3. ****판단 근거****: 각 이슈마다 왜 그 등급으로 분류했는지 한 줄로 남긴다.
4. ****판단 보류****: 요약만으로 심각도를 가늠할 근거가 부족하면 등급을 매기지 말고 "판단 보류"로 표시하고, 무엇이 부족한지(예: "영향 범위 불명", "재현 조건 불명") 한 줄로 남긴다.

절대 하지 말아야 할 것

- 도구를 호출하려 하지 않는다. 추가 정보가 필요해도 조회할 수 없으니, 그 경우 판단 보류로 처리한다.
- 근거 없이 등급을 추측하지 않는다. 애매하면 낮은 확신을 그대로 드러내거나 판단 보류로 남긴다.
- 원본 요약 내용을 왜곡하거나 과장하지 않는다.

출력 형식

등급별로 그룹핑해서 나열한다:

```
...
```

긴급

- [대표 링크] <이슈 한 줄 설명>
- 근거: <한 줄>
- (중복: <링크1>, <링크2>)

높음

도구는 담당마다 따로 지정해야 합니다. "필요한 것만"이라고 하면 넓게 잡힙니다.

도구 권한으로 울다리 치기

무엇을 줄지보다 무엇을 주지 않을지가 중요합니다

에이전트	줄 도구	주지 말아야 할 것
doc-scanner	문서 검색 · 읽기	쓰기 권한 전부
issue-analyst	없음 (받은 내용만 판단)	외부 조회 전부
jira-reporter	Jira 이슈 생성	삭제 · 상태 변경 · 댓글

만들어진 에이전트 확인하기

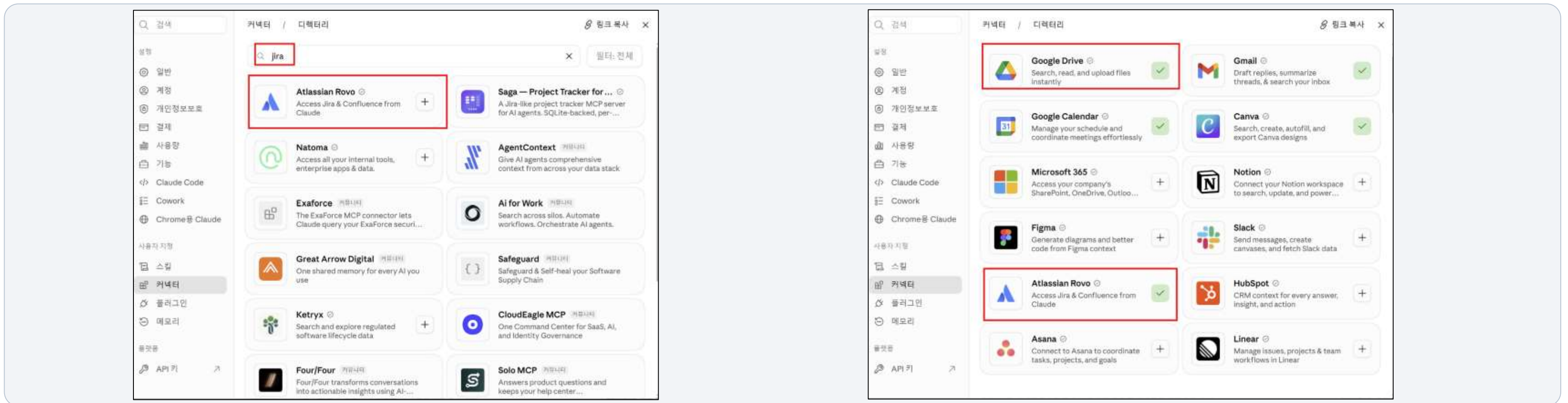
Code 세션 입력칸에 아래 한 줄을 그대로 넣으세요 — 일반 채팅창이 아닙니다

.claude/agents 폴더에 만들어진 파일 목록과 각각의 이름 · 설명 · 쓸 수 있는 도구를 표로 보여줘.

Listed files in .claude/agents directory >			
파일	이름	설명	쓸 수 있는 도구
doc-scanner.md	doc-scanner	Confluence와 구글 드라이브를 검색해 이슈 후보 문서를 찾아낸다. 심각도 판단 없이 원문 링크 + 3 줄 요약만 반환.	Atlassian: getAccessibleAtlassianResources, search, searchConfluenceUsingCql, getConfluenceSpaces, getPagesInConfluenceSpace, getConfluencePage, getConfluencePageDescendants, getConfluencePageFooterComments, getConfluencePageInlineComments, getConfluenceCommentChildren / Google Drive: search_files, list_recent_files, read_file_content, download_file_content, get_file_metadata — 전부 읽기 전용, 쓰기 도구 없음
issue-analyst.md	issue-analyst	doc-scanner 결과의 중복을 제거하고 심각도 4단계(긴급/높음/보통/낮음)로 분류, 근거 한 줄씩 남기고 애매하면 판단 보류	없음 (tools: [])
jira-reporter.md	jira-reporter	분류 결과를 Jira 이슈로 등록. 등록 전 표로 계획 제시 후 승인 받아야 진행	getVisibleJiraProjects, getJiraProjectIssueTypesMetadata, getJiraIssueTypeMetaWithFields, lookupJiraAccountId (조회용) + createJiraIssue (유일한 쓰기 동작). 삭제·상태변경·댓글 도구 없음

커넥터 연결하기 — Atlassian Roovo · 구글 드라이브

설정 → 커넥터 → 디렉터리 → 검색창에 jira → Atlassian Roovo 의 + 버튼



이름 옆에 파란 인증 배지가 있는 것만 고릅니다. "커뮤니티" 표시는 제3자가 만든 것입니다.
구글 드라이브는 검색창에 drive 로 찾아 같은 방법으로 연결합니다.

사이트를 고르는 화면이 나옵니다. 방금 만든 내 사이트인지 반드시 확인하세요.
목록에 회사 사이트가 보이더라도 절대 선택하지 마세요.(테스트중)

문서를 뒤껴 Jira로 보내기

세 명을 한 줄로 세웁니다



Jira 연결은 화면으로 함께 봅니다

회사에서 이미 Jira를 쓰고 계시지만, 오늘은 사내 인스턴스에 연결하지 않습니다

무엇을	어떻게
여러분의 실습 재료	배포해 드린 문서 7건 — 붙여넣기만 하면 됩니다
강사 화면의 문서	같은 7건을 구글 드라이브 폴더 두 곳에 나눠 두었습니다
강사 화면의 Jira	개인 무료 사이트 — 사내 인스턴스가 아닙니다
여러분의 사내 적용	39번에서 순서를 정리해 드립니다

- 사내 Atlassian을 쓰지 않는 세 가지 이유
- 하나. 온프레미스(Data Center)는 공식 MCP가 지원되지 않습니다
- 둘. 실제 프로젝트에 실습용 이슈가 그대로 쌓입니다
- 셋. 사내 시스템에 AI 접근 권한을 여는 것은 보안 검토를 거칠 일입니다

◆ 세션 ② 열고 — 직접 돌려 봅시다

새 창을 하나 더 엽니다. 폴더는 아까와 같은 것을 고르세요

◆ Code → New session → Local → 아까 그 작업 폴더
그다음 배포 파일의 문서 7건을 통째로 복사해 붙여 넣습니다

① 먼저 찾아오게 합니다

위 문서들을 doc-scanner의 방식대로 정리해줘.

문서마다 출처 링크와 3줄 요약만 적고, 심각도는 판단하지 마.

② 결과가 나오면, 이어서 판단하게 합니다

위 결과를 issue-analyst의 방식대로 처리해줘.

같은 사건을 다룬 문서는 하나로 묶어줘.

심각도를 4단계로 나누고 판단 근거를 한 줄씩 남겨줘.

판단할 수 없는 것은 지어내지 말고 '판단 보류'로 남겨줘.

문서는 일곱 건인데 결과가 여섯 건이면 잘 된 것입니다.

하나는 중복이라 묶였고, 하나는 판단 보류로 남습니다.

에이전트가 일하는 동안 무엇을 보나

결과보다 과정을 봐야 다음에 고칠 곳이 보입니다

화면에서 볼 것	왜 봐야 하나
어떤 에이전트가 불렸는지	설명(description)이 의도대로 쓰였는지
각자 도구를 몇 번 썼는지	검색 범위가 너무 넓거나 좁지 않은지
무엇을 반환했는지	요약이 다음 담당에게 충분한지
어디서 멈췄는지	권한이 없어서인지 검색이 실패해서인지

로그는 결과를 의심할 때 보는 곳이 아니라, 다음에 무엇을 고칠지 찾는 곳입니다.

실패해도 멈추지 않게 만들기

프롬프트에 이 다섯 줄을 더합니다

한 소스에서 결과가 없으면 없다고만 적고 다음 소스로 넘어가줘.
검색이 실패하면 두 번까지 다시 시도하고,
계속 실패하면 실패한 소스를 결과 맨 아래에 적어줘.
판단할 수 없는 항목은 지어내지 말고 '판단 보류'로 남겨줘.
등록이 일부만 성공하면 성공한 것과 실패한 것을 나눠서 알려줘.

에이전트를 계속 쓰는 사람과 한 번 쓰고 마는 사람의 차이가 이 다섯 줄에 있습니다.

승인은 위임되지 않습니다

방금 등록 단계에서 한 번 막혔습니다. 버그가 아니라 안전장치입니다

이렇게 시키면	어떻게 되나
jira-reporter에게 등록까지 맡긴다	승인 단계에서 멈춥니다. "사용자가 승인했다"는 전달을 인정하지 않습니다
목록만 받고 내가 승인한 뒤 메인 대화에서 등록한다	정상 등록됩니다

만약 "사용자가 승인했다"는 말을 그대로 믿어 준다면,
문서에 그 한 문장을 심어 두는 것만으로 승인을 위조할 수 있습니다.

사람이 있는 자리에서만 승인이 됩니다. 이것이 마지막 안전장치입니다.

Jira에 등록된 결과 확인하기

승인한 항목만 이슈로 올라와 있습니다

KAN 프로젝트로 등록 계획을 세우겠습니다. 이전에 작업하던 jira-reporter 에이전트를 이어서 진행합니다.

실행됨 에이전트 1개, 사용함 도구 1개 >

KAN 프로젝트에 등록할 계획입니다. 이슈 타입은 "버그"가 없어서 "작업"으로, 우선순위는 긴급→Highest, 높음→High로 매핑됩니다.

#	프로젝트	이슈 타입	제목	심각도	원문 링크
1	KAN	작업	[긴급] 08-24 아시아 리전 로그인 실패율 급등(0.3%→18%) - 인증 토큰 서버 메모리 누수	Highest	문서 링크
2	KAN	작업	[높음] 결제 취소 API 성공 응답에도 클라이언트 잔액 화면 미반영	High	문서 링크
3	KAN	작업	[높음] 로그인 응답 시간 최근 2주간 380ms→620ms 증가(21~23시 집중)	High	문서 링크

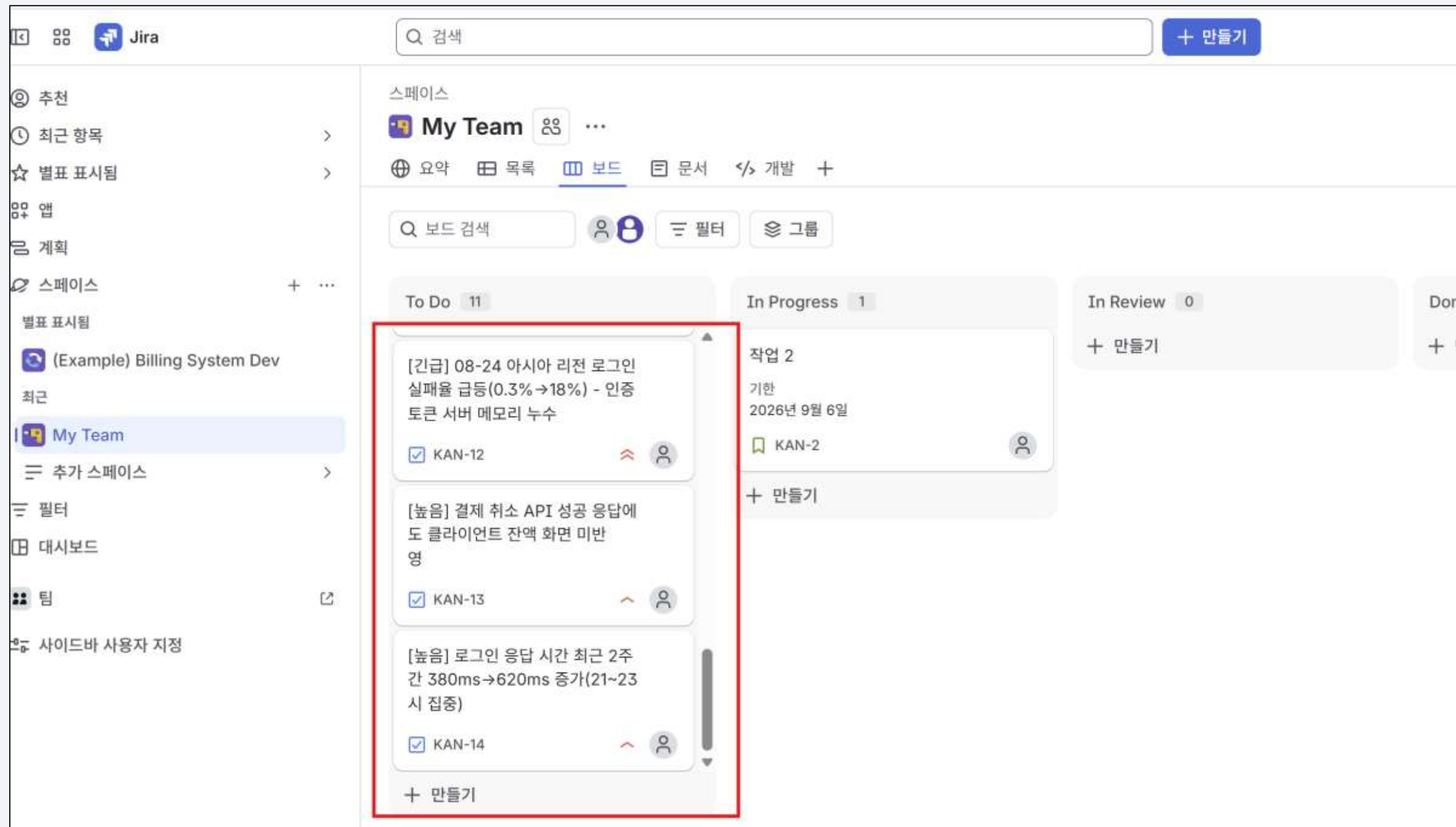
이대로 3건을 KAN에 등록해도 될까요? (승인해주시면 진행합니다.)

승인

판단 근거가 이슈 본문에 함께 남아 있는지 확인하세요.

Jira에 등록된 결과 확인하기

승인한 항목만 이슈로 올라와 있습니다



Jira에 등록된 결과 확인하기

승인한 항목만 이슈로 올라와 있습니다

epic 추가 / ☒ KAN-12

[긴급] 08-24 아시아 리전 로그인 실패율 급등(0.3%→18%) - 인증 토큰 서버 메모리 누수

+

...

설명

08-24 21:14~21:54 아시아 리전에서 로그인 실패율이 0.3%에서 18%로 급등. 원인은 인증 토큰 서버의 메모리 누수. 서비스 중단급 장애 발생, 재발방지 조치(메모리 알림 임계값 조정) 아직 미적용. 동일 시간대 커뮤니티에서도 로그인 불가 반응 확인됨. 관련 문서:
https://docs.google.com/document/d/1gZZLWXs8aJqwBMoUCKF6fQncQah5jAWMv_1ZXBEtIoo/edit
원문 링크: <https://docs.google.com/document/d/16A2HmXhFS8agn7FBmZ790KjlfIBRD6fUJaylvgsqkU/edit>

하위 작업

하위 작업 추가

연결된 업무 항목

연결된 업무 항목 추가

1

해야 할 일

업무 유형 작업 개선

세부 사항

담당자	할당되지 않음 나에게 할당
상위 항목	없음
우선 순위	Highest
레이블	없음
기한	없음
Team	없음
시작 날짜	없음
보고자	전혜영

매일 알아서 돌게 만들기

오전 9시에는 아무도 없습니다. 그래서 등록은 사람이 아침에 합니다

언제	누가	무엇을
오전 9시	예약이 있어서	문서 조사 → 중복 제거 → 심각도 분류 → 목록 작성까지
출근해서	여러분이	만들어져 있는 목록을 보고 등록할 것을 고릅니다
그 자리에서	여러분이	한 줄 말하면 승인 화면이 뜨고, 승인하면 등록됩니다

한 번만 — 예약 만들 때

매일 오전 9시에 조사와 분류까지만 해줘.
등록은 하지 말고 목록만 보여줘.

매일 아침 — 등록할 때

오늘 아침 예약 결과에서
긴급·높음만 Jira에 등록해줘.

이 예약은 앱이 켜져 있는 동안에만 돕니다. 노트북을 닫으면 그 시간에 돌지 않습니다.

팀에 전달하고 안전하게 쓰기

내 것으로 끝나면 절반입니다



한 번 만들고 끝이 아닙니다

결과가 이상하면 프롬프트를 다시 쓰지 말고, 정의 파일을 고칩니다

이런 일이 생기면	어디를 고치나	이렇게 말하면 됩니다
엉뚱한 때 불리거나 아예 안 불린다	description (위쪽 설명 줄)	"doc-scanner의 설명을 언제 불려야 하는지 분명하게 고쳐줘"
판단 기준이 내 생각과 다르다	업무 지침 (아래쪽 본문)	"issue-analyst에 이 기준을 추가하고 다시 처리해줘"
쓰지 말아야 할 도구를 쓴다	tools (도구 목록)	"jira-reporter는 이슈 생성만 쓸 수 있게 해줘"

고친 정의 파일은 저장소에 함께 두세요.
누가 언제 무엇을 왜 바꿨는지가 남아야 팀이 같이 쓸 수 있습니다.

고치는 데 코드가 필요 없습니다. 말로 고치고 바로 다시 돌려 봅니다.

플러그인으로 묶어 팀에 전달하기

각자 만들면 각자 다르게 동작합니다

방법	어떻게	어울리는 상황
플러그인 + 사내 저장소	마켓플레이스로 등록하고 설치	팀 전원이 같은 구성을 쓸 때
프로젝트 폴더에 함께 두기	.claude/agents를 저장소에 커밋	특정 저장소에서만 쓸 때
각자 만들기	각자 알아서	권하지 않습니다

지금 만든 에이전트 3개를 플러그인으로 묶어줘.

한 사람이 만들어 열다섯 명이 같은 것을 씁니다. 인프라 비용은 들지 않습니다.

SDK를 설계하듯 에이전트를 설계합니다

남이 쓸 것을 만든다는 점에서 같습니다

SDK를 설계할 때	에이전트를 설계할 때
공개 인터페이스를 좁게 잡는다	쓸 수 있는 도구를 좁게 잡는다
기본값이 안전하도록 정한다	기본은 승인을 받고 실행하게 한다
버전과 변경 이력을 남긴다	정의 파일을 저장소에 함께 둔다
문서가 없으면 아무도 안 쓴다	설명이 부실하면 아무 때도 안 불린다

여러분이 SDK를 설계할 때 고민하는 것과 같은 문제입니다.

권한을 주면 생기는 일

실제로 있었던 일들입니다

시점	무슨 일이 있었나
2025. 5. GitHub MCP	공개 저장소에 올라온 악성 이슈 본문이 어시스턴트를 움직여, 비공개 저장소의 내용을 공개 PR로 내보냈습니다
2026. 4. STDIO 커맨드 인젝션	널리 쓰이는 MCP 서버들에서 CVE 10건. 누적 다운로드 1.5억 건이 영향 범위에 들었습니다
2026. 2. 악성 클론	공개 레지스트리에 올라온 가짜 MCP 서버가 연결한 계정의 자격증명을 탈취했습니다

에이전트가 읽는 외부 텍스트는 전부 명령이 될 수 있습니다.
문서도, 이슈 본문도, 댓글도.

출처: Invariant Labs (2025. 5.) · ox.security (2026. 4.)

우리 팀에서는 어디까지 허용할까

오른쪽 칸을 각자 채워 보세요

단계	이런 작업	우리 팀은?
읽기만	문서 검색, 이슈 조회, 요약	
승인 후 쓰기	이슈 등록, 댓글 작성, 상태 변경	
자동 쓰기	예약 실행으로 사람 확인 없이 등록	

처음에는 전부 가운데 칸에 둡니다. 한 달 돌려 보고 하나씩 아래로 내립니다.

빈칸을 채우면 됩니다

내 업무를 세 조각으로 자르는 연습입니다

[에이전트 팀 설계]

- 내가 매주 반복하는 조사 업무 : []
- 자료가 흩어져 있는 곳 : [] [] []
- 나눌 역할
 - 수집 담당 - 어디서 무엇을 : []
 - 판단 담당 - 무슨 기준으로 : []
 - 기록 담당 - 어디에 어떤 형식으로 : []
- 자동으로 해도 되는 것 / 승인이 필요한 것 : []
- 실행 주기 : []

위 내용으로 서브에이전트를 만들어줘. 각자 필요한 도구만 쓰게 해주고, 기록 담당은 실행 전에 반드시 내 승인을 받게 해줘.

무엇을 채울지 모르겠다면

이미 하고 있는 일을 세 조각으로 자르는 것입니다

이번 주에 이런 일이 있었다면	그게 이 담당입니다
같은 것을 두 번 이상 찾아봤다	수집 담당
찾고 나서 급한지 아닌지 스스로 판단했다	판단 담당
그 결과를 어딘가에 옮겨 적었다	기록 담당

새로운 업무를 만드는 것이 아닙니다. 이미 하고 있는 일을 세 조각으로 자르는 것입니다.

토큰을 아끼면서 오래 쓰기

나누는 방식은 토큰을 더 쓰기 때문에 관리가 필요합니다

이렇게 하면	이런 효과가 있습니다
서브에이전트의 tools를 좁힌다	도구 정의만큼 컨텍스트가 줄어듭니다
커넥터는 쓸 때만 켜다	여럿 붙이면 대화를 시작하기도 전에 토큰이 나갑니다
긴 작업은 예약으로 돌린다	결과만 받아 보면 됩니다
대화가 길어지면 요약해 새로 시작	매번 다시 읽는 양을 초기화합니다

나누는 설계는 토큰을 더 쓰는 대신, 각자가 읽는 양을 줄여 되돌려 받습니다.

전체 흐름 한눈에 보기

오늘 만든 것이 이렇게 이어집니다



찾고 판단하는 일은 나눠서 맡기고, 승인은 사람이 합니다.
이 경계를 어디에 두느냐가 오늘 배운 전부입니다.

이틀 동안 만든 것

오늘 가져가는 것은 세 가지입니다

가져가는 것	무엇인가
에이전트 정의 파일 3개	수집 · 판단 · 기록 담당. 폴더째 복사하면 어디서든 됩니다
매일 도는 예약 작업	아침마다 결과를 먼저 받아 보는 구조
플러그인 하나	팀에 그대로 설치하게 할 수 있는 패키지

오늘 만든 세 명은 그대로 두고, 이름과 지시만 바꾸면 다른 업무가 됩니다.

월요일부터 할 수 있는 것

오늘 만든 세 파일은 다시 만들 필요가 없습니다

언제	무엇을	승인
월요일 바로	구글 드라이브·독스에서 자료를 찾아 정리하는 에이전트	필요 없음
월요일 바로	역할 나누기 · 도구 권한 좁히기 · 승인 걸기	필요 없음
몇 주 뒤	사내 Jira·Confluence 커넥터 — 읽기 전용부터	보안팀 승인
그 다음	플러그인으로 팀 전체에 배포	승인 + 배포

세 파일에는 어느 사이트에 붙일지가 적혀 있지 않습니다. 커넥터가 그것을 정합니다.
회사 사이트로 다시 연결하면, 파일을 고치지 않아도 그대로 회사 Jira에 등록합니다.

오늘 만든 것은 실습용 사본이 아니라, 연결만 바꾸면 되는 완성품입니다.

회사에 돌아가서 혼자 하는 순서

이 여섯 줄을 위에서부터 그대로 하시면 됩니다

	하는 일	실제로 이렇게 하세요
1	세션 열기	Claude Desktop → Code → New session → Local → 작업 폴더 선택
2	에이전트 넣기	오늘 받아 가신 완성본의 .claude 폴더를 그 작업 폴더에 복사
3	커넥터 연결	설정 → 커넥터 → 디렉터리 → 검색 → + → 브라우저에서 승인
4	이름 맞추기	"연결된 커넥터의 실제 도구 이름으로 tools를 고쳐줘"
5	돌려 보기	"○○를 조사해서 등록할 목록을 표로 보여줘"
6	예약 걸기	"매일 오전 9시에 위 순서를 그대로 실행해줘"

4번을 빼먹으면 에이전트가 아예 불리지 않습니다
등록까지 한 번에 맡기지 마세요. 목록 → 내 승인 → 등록 순서입니다

오늘 직접 못 해 본 것: 3번

클라우드 토큰 절약 전략



왜 클로드는 토큰이 금방 소진될까?

C L A U D E

토큰 절약
완전 정복

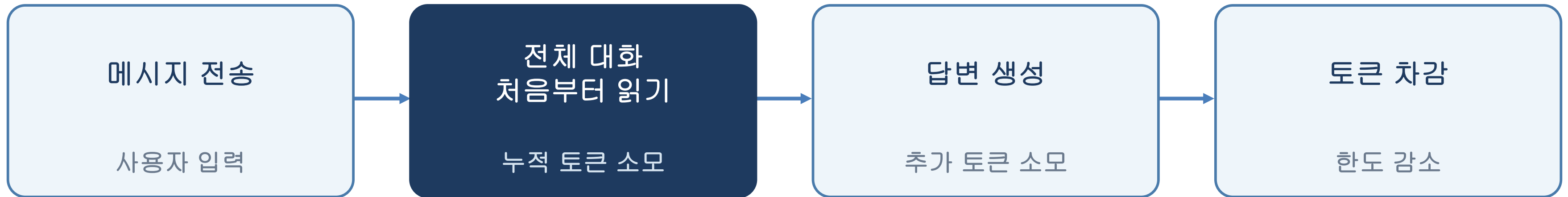
꿀팁 11가지

📖 매 메시지마다 대화 전체를 처음부터 다시 읽음

📈 토큰 소모: 100 → 200 → 400 → 800 (기하급수 증가)

⚠️ 메시지 100개 초과 시 토큰의 98.5%가 이전 대화 읽기에 소모

클로드 토큰 작동 원리



토큰이란?

AI가 텍스트를 처리하는 최소 단위
단어 1개 $\approx$ 1.3 토큰

맥락 왜곡 현상

대화가 너무 길어지면
토큰이 분산되어 AI 성능 저하

누적 증가 공식

대화 1회: 100토큰

대화 2회: 200토큰

대화 4회: 400토큰 (기하급수)

메시지 100개 초과 시

98.5% $\rightarrow$ 이전 대화 읽기

1.5%만 $\rightarrow$ 실제 작업에 사용

대화 관리 전략

01

Edit 기능

채팅 수정하기

답변이 마음에 안 들 때 →
후속 메시지 ✕
내 메시지에 마우스 올려
연필 아이콘 클릭
→ 원본 프롬프트 수정 후 재전송 ✓

💡 잘못된 답변이 대화 기록에
쌓이지 않아 불필요한 토큰 차단

02

New Chat

채팅 새로 시작하기

메시지 15~20개 전후로 새 채팅 시작
전환 전: "지금 대화를 마크다운으로
요약해줘" 요청
새 채팅에 요약본 첨부 후 이어서 작업

💡 기하급수적 토큰 증가를 초기화하여
효율 극대화

03

Batch Request

메시지 묶어 보내기

질문을 하나씩 따로 보내기 ✕
여러 요청을 하나의 프롬프트에 통합 ✓
예: 요약 → 인사이트 추출 →
보고서 작성 한 번에 요청

💡 채팅 교환 횟수를 줄여
누적 토큰 증가 방지

파일 & 설정 최적화

04 프로젝트 기능 활용

- 파일을 프로젝트에 한 번만 업로드
- 모든 채팅에서 자동 공유됨
- 채팅마다 파일 재첨부 불필요
- 프로젝트별 맞춤 지침도 설정 가능

05 마크다운 파일 활용

70% ↓

PDF → Markdown
토큰 절감

90% ↓

HTML → Markdown
토큰 절감

06 개인 맞춤 설정 활용

전역 설정

- 매번 역할 부여·지침 입력 ✕
- 설정 → 개인 맞춤 설정에 공통 지침 등록 ✔
- 모든 채팅에 자동 적용됨

프로젝트별 설정

- 유튜브 기획 프로젝트 → '너는 유튜브 전문 기획자야'
- 인스타 콘텐츠 프로젝트 → '너는 인스타 전문 마케터야'
- 카테고리별 역할을 따로 지정 가능

기능 & 모델 최적화

07 필요할 때만 기능 켜기

켜져 있으면 토큰 소모되는 기능들:

 웹 검색 기능

 외부앱 커넥터

☐ 적응형 사고 기능

💡 사용 전: ON → 사용 후: 즉시 OFF
PDF 요약 시 웹검색 필요 없음 → 꺼두기!

08 모델 구분해서 쓰기

Haiku

경량

간단한 질문
가벼운 작업
빠른 속도

Sonnet

기본

일반 업무
중간 난이도
균형형 모델

Opus

고성능

복잡한 추론
고난이도 분석
토큰 많이 소모

시간 & 비용 & 분산 관리

09 피크 시간 피하기

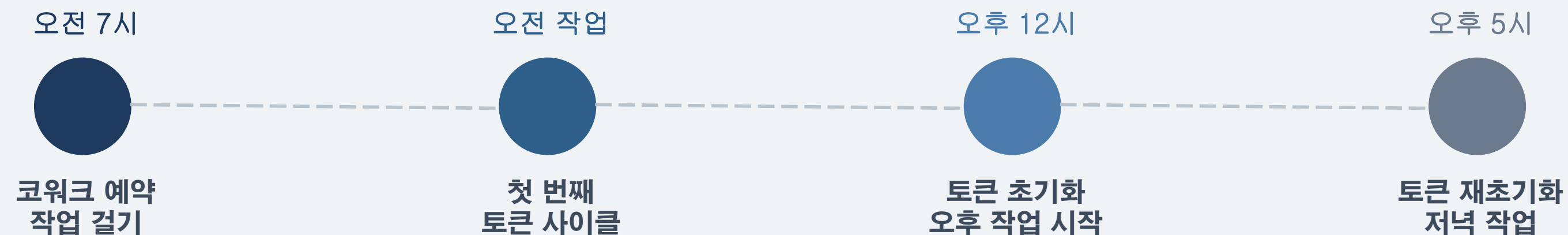
미국 동부	오전 8시 ~ 오후 2시 (피크)
한국 시간	밤 10시 ~ 새벽 4시
서머타임	밤 9시 ~ 새벽 3시
추천 시간	평일 낮 / 주말 ← 최적! ✓

10 추가 사용량 기능

- 설정 → 사용량 탭 → 추가 사용량 활성화
- 한도 초과 후에도 작업 계속 가능
- 월간 지출 한도 설정으로 요금 폭탄 방지
- 선불 충전 방식도 선택 가능

11 작업 분산하기 🕒 꿀팁

토큰은 사용 시작 시점부터 5시간마다 초기화!



3D 역개발



마음에 드는 화면을 그대로 만드는 법

3D를 몰라도 됩니다. 순서 세 개만 기억하면 됩니다

	무엇을 한다	어떻게
1	마음에 드는 화면을 찾는다	웹사이트든 앱이든, 스크린샷 한 장이면 됩니다
2	AI에게 분석시킨다	"이 화면을 만들려면 뭐가 필요한지 분석해서 프롬프트로 정리해줘"
3	뒤에 한 줄만 붙인다	"이걸로 우리 팀 소개 화면 만들어줘"

2번에서 나온 프롬프트에는 모르는 단어가 가득할 겁니다.
하나도 몰라도 됩니다. 읽지 말고 그대로 복사해서 쓰면 됩니다.

여러분이 직접 쓰는 것은 3번 한 줄뿐입니다.

마음에 드는 화면을 그대로 만드는 법

3D를 몰라도 됩니다. 순서 세 개만 기억하면 됩니다

	무엇을 한다	어떻게
1	마음에 드는 화면을 찾는다	웹사이트든 앱이든, 스크린샷 한 장이면 됩니다
2	AI에게 분석시킨다	"이 화면을 만들려면 뭐가 필요한지 분석해서 프롬프트로 정리해줘"
3	뒤에 한 줄만 붙인다	"이걸로 우리 팀 소개 화면 만들어줘"

2번에서 나온 프롬프트에는 모르는 단어가 가득할 겁니다.
하나도 몰라도 됩니다. 읽지 말고 그대로 복사해서 쓰면 됩니다.

여러분이 직접 쓰는 것은 3번 한 줄뿐입니다.

마지막 한 줄만 바꿨습니다

앞부분은 똑같습니다. 바꾼 것은 맨 끝 한 줄뿐입니다

맨 끝에 붙인 한 줄	나온 결과
중세 마을을 배경으로 만들어줘	성과 집이 있는 3D 마을
공중에 떠 있는 섬으로 만들어줘	섬과 계단이 떠 있는 화면
우리 사무실을 3D로 만들어줘	층별 좌석을 둘러보는 화면

직접 보고 가실 곳 — 설치도 로그인도 없이 주소만 치면 열립니다
expeditione.fun · mopedmayhem.io

앞부분은 AI가 써 준 것을 복사한 것입니다. 여러분이 쓴 것은 마지막 한 줄입니다.

마지막 한 줄만 바꿨습니다

앞부분은 똑같습니다. 바꾼 것은 맨 끝 한 줄뿐입니다

AI 통해서 분석시켜서 요약하면 .아래와 같은 프롬프트가 나온다.

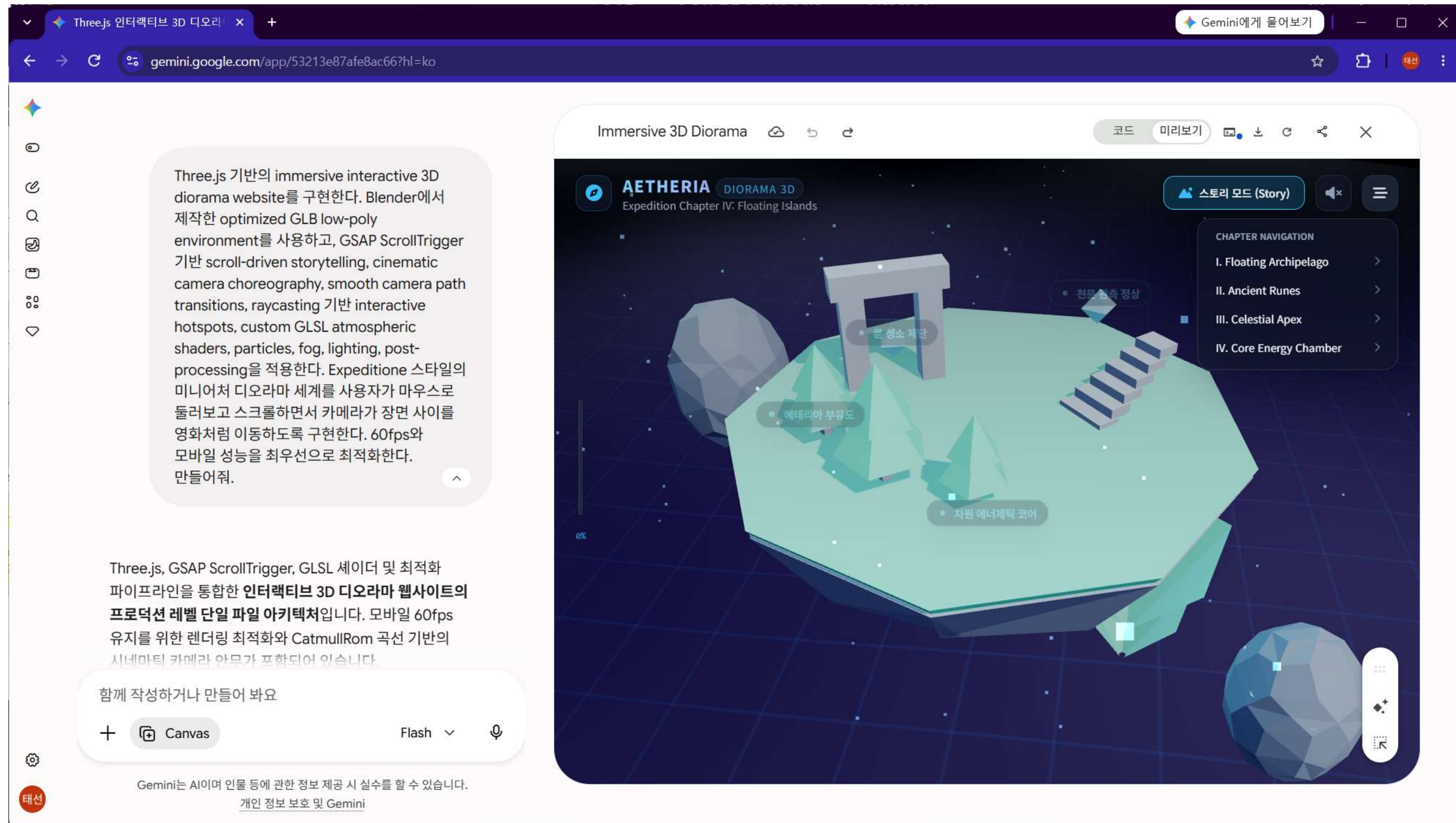
Three.js 기반의 immersive interactive 3D diorama website를 구현한다. Blender에서 제작한 optimized GLB low-poly environment를 사용하고, GSAP ScrollTrigger 기반 scroll-driven storytelling, cinematic camera choreography, smooth camera path transitions, raycasting 기반 interactive hotspots, custom GLSL atmospheric shaders, particles, fog, lighting, post-processing을 적용한다.

뒤에 원하는 걸 적으면 된다.

Three.js 기반의 immersive interactive 3D diorama website를 구현한다. Blender에서 제작한 optimized GLB low-poly environment를 사용하고, GSAP ScrollTrigger 기반 scroll-driven storytelling, cinematic camera choreography, smooth camera path transitions, raycasting 기반 interactive hotspots, custom GLSL atmospheric shaders, particles, fog, lighting, post-processing을 적용한다. **Expedition 스타일의 미니어처 디오라마 세계를 사용자가 마우스로 둘러보고 스크롤하면서 카메라가 장면 사이를 영화처럼 이동하도록 구현한다. 60fps와 모바일 성능을 최우선으로 최적화한다.**

마지막 한 줄만 바꿨습니다_Gemini/Canvas

앞부분은 똑같습니다. 바꾼 것은 맨 끝 한 줄뿐입니다



하나 골라서 직접 만들어 보세요

앞의 프롬프트 뒤에 이 한 줄만 바꿔 붙이면 됩니다

만들 것	뒤에 붙일 한 줄
우리 팀 지표 한 화면	일별 접속자·결제·오류 건수를 카드로 보여주는 대시보드로 만들어줘
결제 흐름 설명 화면	결제 요청부터 완료까지 단계를 스크롤에 따라 하나씩 보여줘
서버 상태 보드	지역별 서버 상태를 지도 위에 점으로 표시하고 클릭하면 상세가 뜨게 해줘
3D 미니 게임	캐릭터가 장애물을 피해 달리는 간단한 웹게임으로 만들어줘
우리 사무실 3D	층별 좌석 배치를 3D로 돌려볼 수 있게 만들어줘

참고사이트: <https://mopedmayhem.io/play?ref=webgpu.com>

<https://www.webgpu.com/>

프롬프트 한 줄로 화면이 나온다는 것만 확인하시면 됩니다.



Thank you

